

基于 YOLO 家族的系统探究

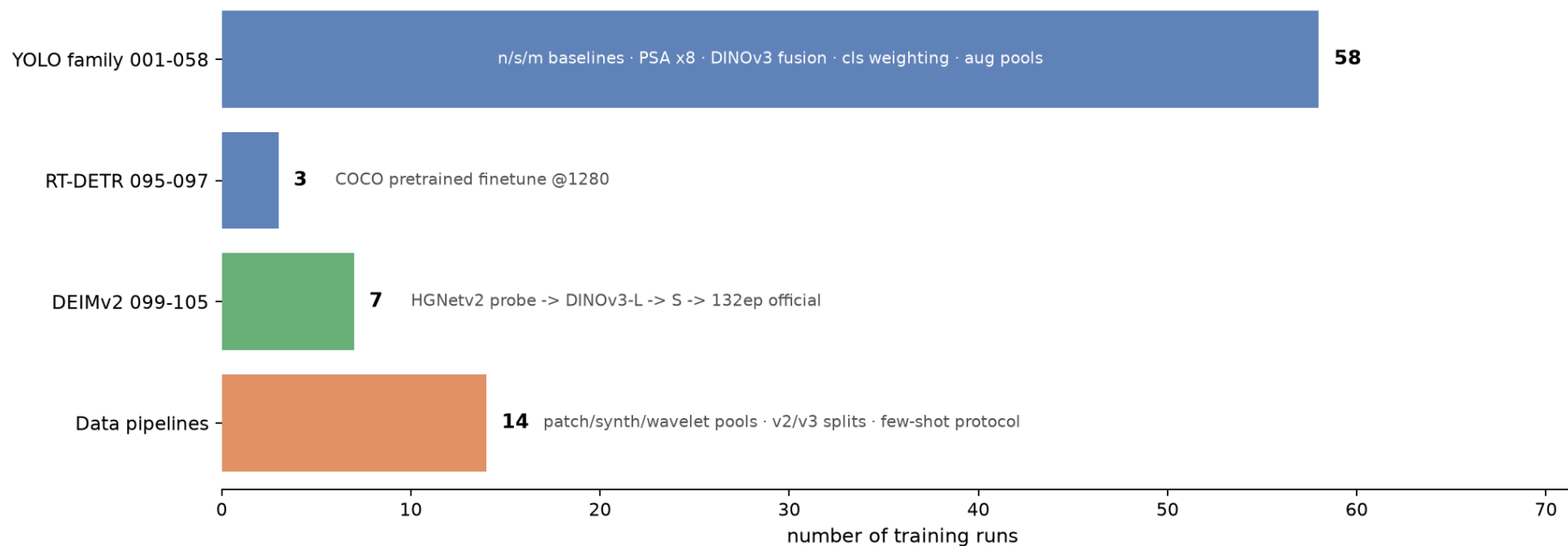
超参数调整 · 架构调整 · 数据集重划分 · 数据增强

81 个训练 run · 6 条数据管线 · 3 套评测协议 · 2026-09

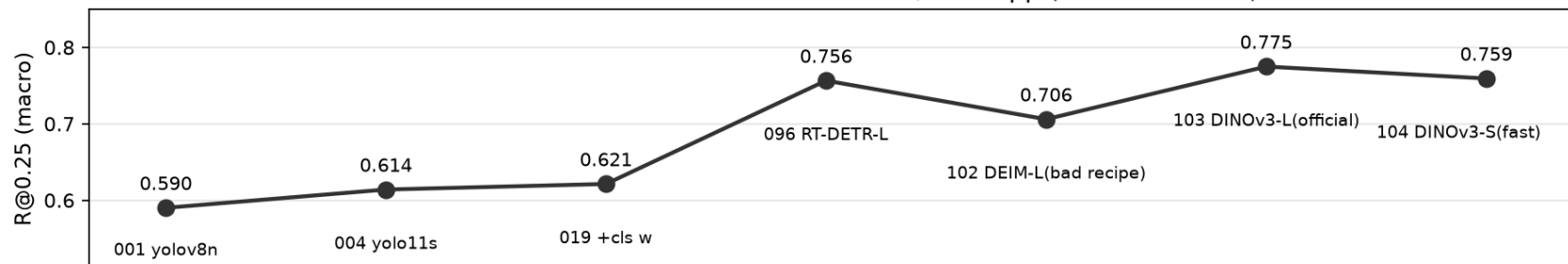
探究框架：五个维度

维度	实验范围	run 数	核心结论
基线选型	yolov8n/s/m · yolo11n/s/m · yolo26 · yoloworld	7	yolo11s 最优 (0.614)
超参数调整	分辨率 · epoch · lr/冻结 · 增强强度 · 类别加权	26	cls 加权唯一正收益；长训/强增强全负
架构调整	PSA/SE 注意力 ×6 · P2 融合 · DINOv3-fusion	13	结构改造全负 → 转向范式
数据集重划分	v1 前缀划分 → v2 类分层 → v3 迭代分层	3 套	泄漏教训：跨集验证作废
数据增强	patch/合成/对比度/小波/放大/背景池	9 池	+10.5pp (063) · 四种池形态证伪

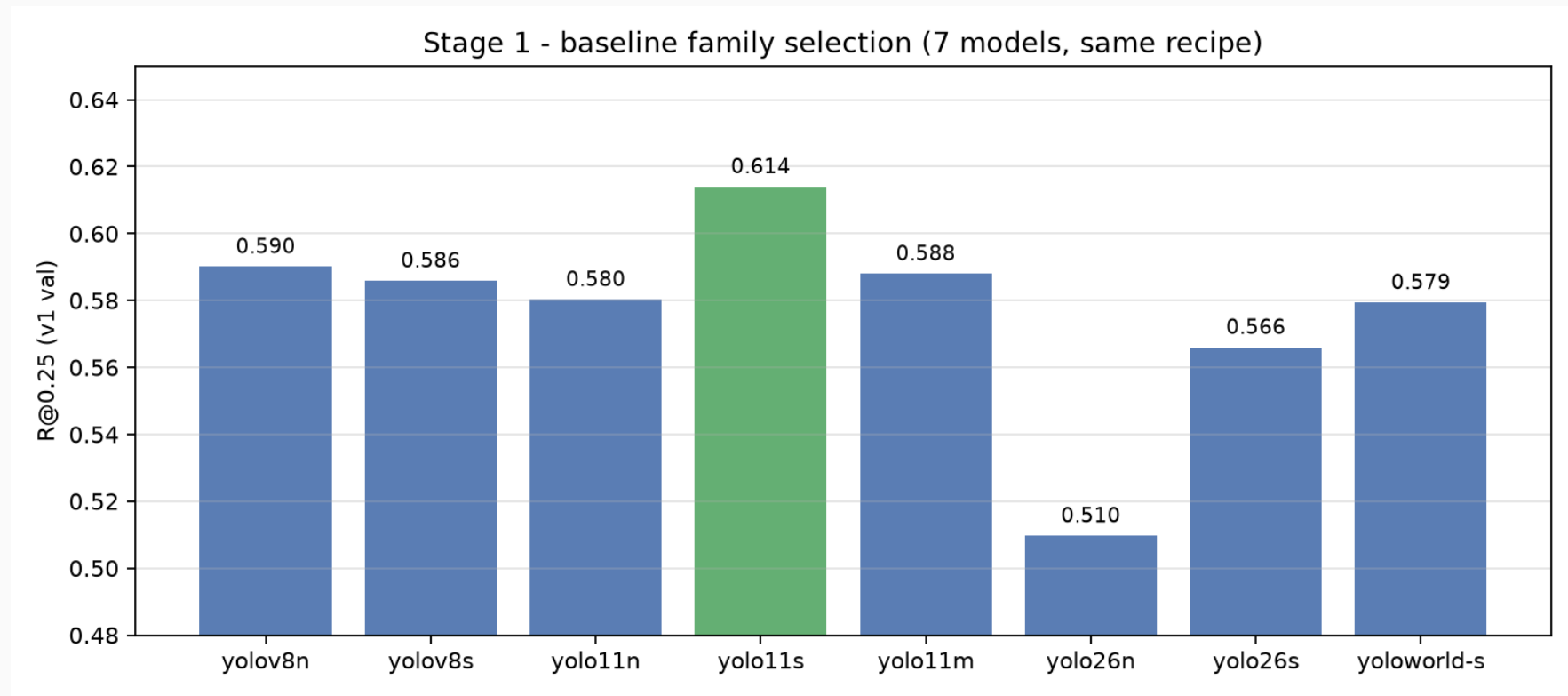
Workload: 81 training runs + 6 data pipelines + 3 eval protocols



Metric evolution: 3 architecture rounds, +11.6pp (0.590 -> 0.775)



维度一 · 基线选型：同配方横向对比



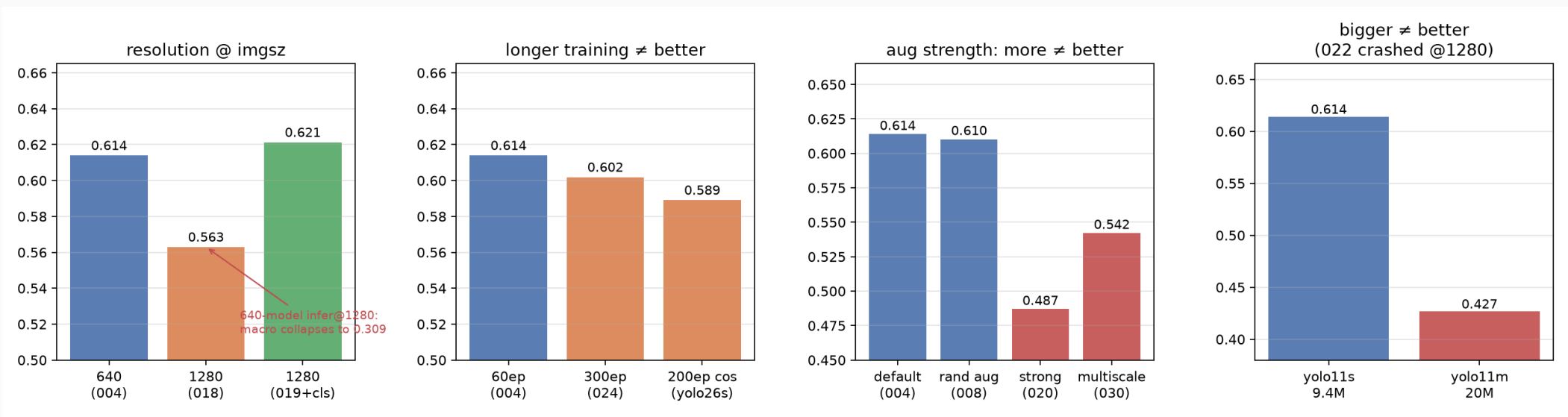
统一控制变量：同数据 (v1 train 1445)、同 epoch、同增强、同 val 口径 (conf 0.25 / IoU 0.5)

发现：

- yolo11s 全面胜出 (0.614) ——特征层级与 17 类缺陷的尺度分布匹配
- yolo26 系 (新架构) 反而垫底 (0.51-0.57) ——默认超参不适配小目标密集场景
- yolo11m 更大模型 @1280 崩溃 (0.427) ——**大模型+大分辨率在小数据上过拟合**

超参数调整 · 分辨率杠杆（最重要也最危险的超参）

超参数调整 · 分辨率杠杆（最重要也最危险的超参）



分辨率：1280 训练本身不涨分（018：0.563），但 640 模型 @1280 推理会崩溃（宏 0.309）—— **训练-推理分辨率必须严格绑定**，这一条约束了后续所有实验设计。 **规模**：yolo11m 参数翻倍反而 -18.8pp（022）——6GB 显存限制下 batch 被迫压小，大模型训练质量崩坏。

超参数调整 · 训练时长 / 冻结 / 增强强度

超参	变体	R@0.25
epoch	60 (默认)	0.614
epoch	300 (×5)	0.602 ↓
epoch	200 cosine (yolo26s)	0.589 ↓
冻结	backbone freeze 10ep	0.534 ↓↓
增强	RandAug	0.610 ≈
增强	strong aug	0.487 ↓↓
增强	multiscale	0.542 ↓

三条超参规律 (58 个 run 归纳):

- 1. 默认超参已接近最优: 60ep/默认增强即最优, 加倍 epoch -1.2pp
 - 原因: 2838 图的小数据集, 长训=过拟合小目标
 - 冻结有害: 小目标需要全网络梯度重塑尺度表征
 - 增强过强有害: strong-aug -12.7pp——小缺陷在强几何变换下被破坏

例外: RandAug 温和档 (008, 0.610) 与基线持平——增强的“剂量”比“种类”重要

超参数调整 · 类别加权 loss（自研补丁）

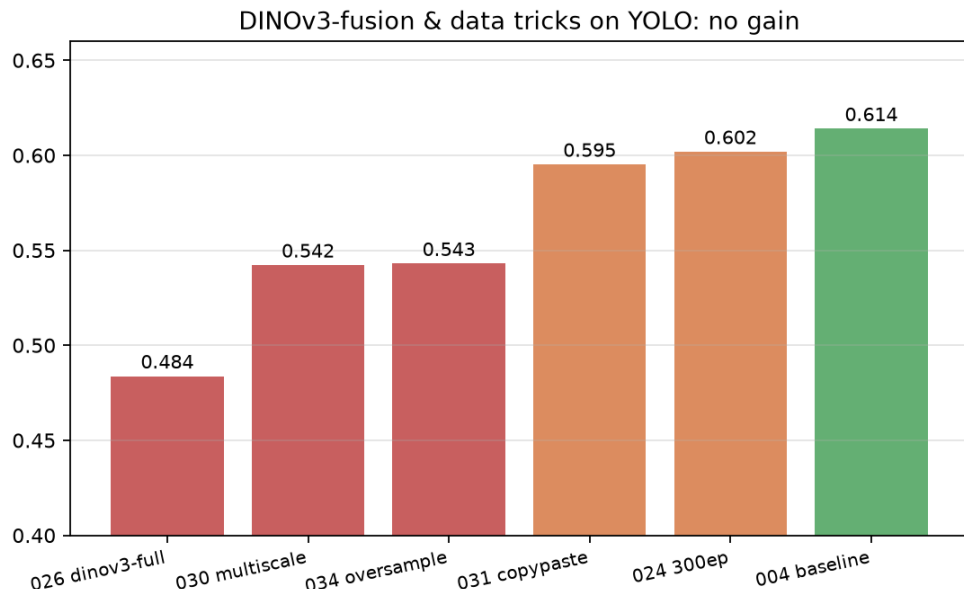
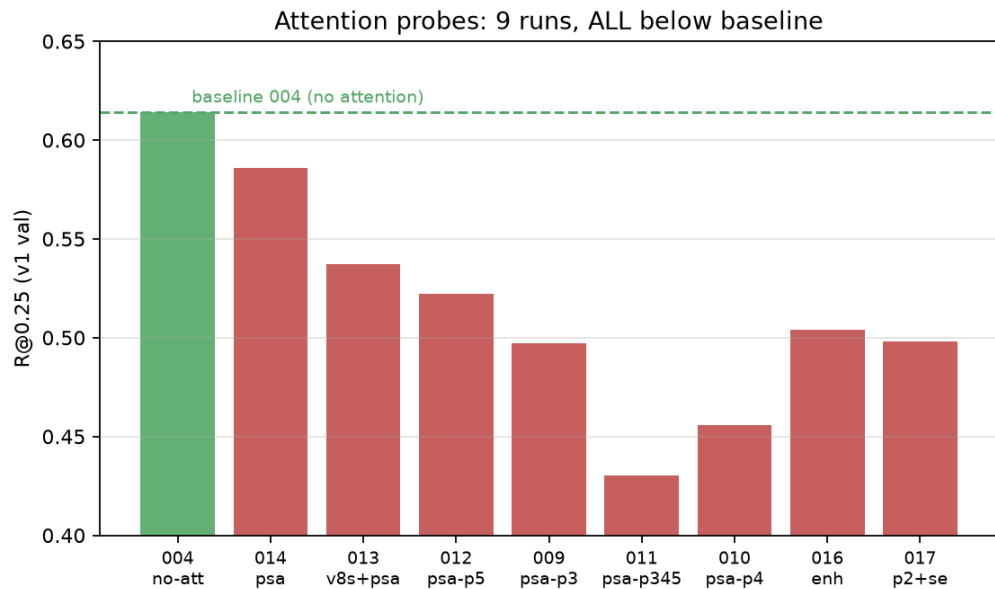
[自研 loss 补丁（fork ultralytics v8DetectionLoss，环境变量门控）：

- SMALL_DEFECT_CLS=1,2 + GAIN=6：bd/heidian 类级 cls 加权
- SMALL_DEFECT_AREA：面积感知（小目标 cls loss ×gain）
- SMALL_DEFECT_NEG/NEG_GAMMA：难例背景加权（ $w = 1+(N-1)p^\gamma$ ）

结果：019 = 0.6213（+0.7pp），bd/heidian 召回显著回升； 但 NEG 公式错误版（066）曾崩溃至 54300 框——每次改动都做了单变量验证]

loss 变体	R@0.25
004 基线	0.614
019 +cls 加权	0.621
029 +clspw 变体	0.580
066 NEG 崩溃案例	训练发散

架构调整 · 注意力机制系统扫描 (9 run 全负)



系统性结论：PSA 插入 P3/P4/P5/P345/全层、C2PSA、SE、P2+深浅 SE——**9 个配置全部低于无注意力基线** (-0.03 -0.18)。**为什么：**2838 张训练图不足以让注意力模块学到额外信号；插注意力还稀释了 P2 小目标特征的梯度。**价值：**一次性排除整个“结构微创新”方向，把算力转向数据与预训练。

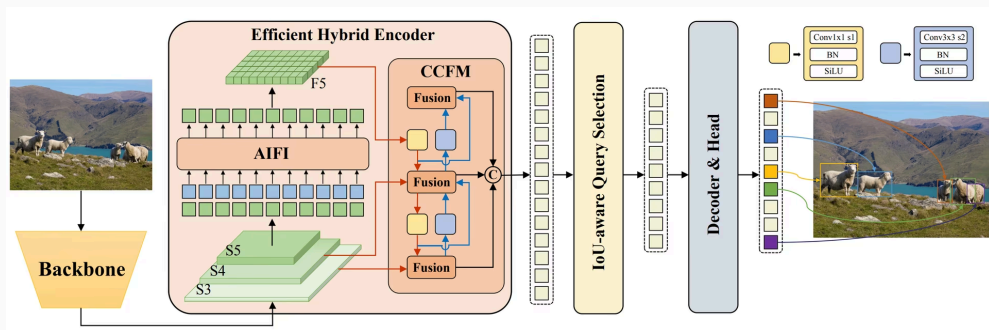
架构调整 · DINOv3 融合进 YOLO（8 run 无增益）

尝试矩阵： dinov3-full backbone / convnext / ViT-MS / hybrid 双主干 / 100ep 长训
（025-041，8 个变体） **全部 ≤ 基线**（最高 0.543-0.596 vs 0.614）

教训： 把预训练表征**拼接**进 YOLO neck，破坏了原特征金字塔的尺度对齐—— 免费特征 ≠ 免费精度。要吃进 DINOv3 表征需要**整个检测头按它设计**（→ 后面 DEIMv2 路线）

Attention structures (PSA/C2PSA/SE)	6 runs (009-017)
all BELOW yolo11s baseline (-0.03 ~ -0.18): attention adds no signal on this data	
Background suppression	4 forms (065/066/067/070)
all negative: hard-neg backgrounds hurt bd/zmty; high FP != pure background (sparse val labels)	
Oversampled / upscaled pools	run 078
11x train-inference scale mismatch -> bd recall drops; scale alignment beats data volume	
Copy-Paste	run 031
0.595 vs 0.614 baseline - no gain on dense small defects	
YOLO-Master / LoRA 069	parked
superseded by data-driven route + cloud training	

架构调整 · 范式转换：YOLO → RT-DETR (+13.5pp)



run	R@0.25
095 @640	0.707
096 @1280 双卡	0.757
097 adamw 显式	0.716

为什么 **transformer** 在小数据上赢：COCO 预训练的检测先验 + 可变形注意力对密集小目标的查询分配——这两个能力在 YOLO 上加注意力模块是补不回来的。**结论：换范式 > 改结构。**

数据集重划分 · 三代划分与泄漏教训

Split evolution (fig-level, class-stratified)

v1	1445/181/- prefix-based first attempt	377 GT	004: 0.614
v2	1447/350/349 class-stratified v2	678 GT	044: 0.532
v3	1448/180/179 8:1:1 iterative stratified	~430 GT	079: 0.584

*different val sets — NOT cross-comparable; v2 val overlapped 80.9% with v1 train (leakage, scrapped)

Leakage audit (the 80.9% lesson)

v2 split reused v1 images:
- v2 val (350 imgs) overlaps v1 train 80.9%
- any v2-val score leaks v1-train knowledge
- all cross-split numbers invalidated

Fixes adopted:
1. fig-level 8:1:1 with iterative class-stratification
2. per-class val ≥ 8 instances check
3. leak-check script on every new split
4. every run archives its data manifest

Rule: never report a metric on a split the model may have seen

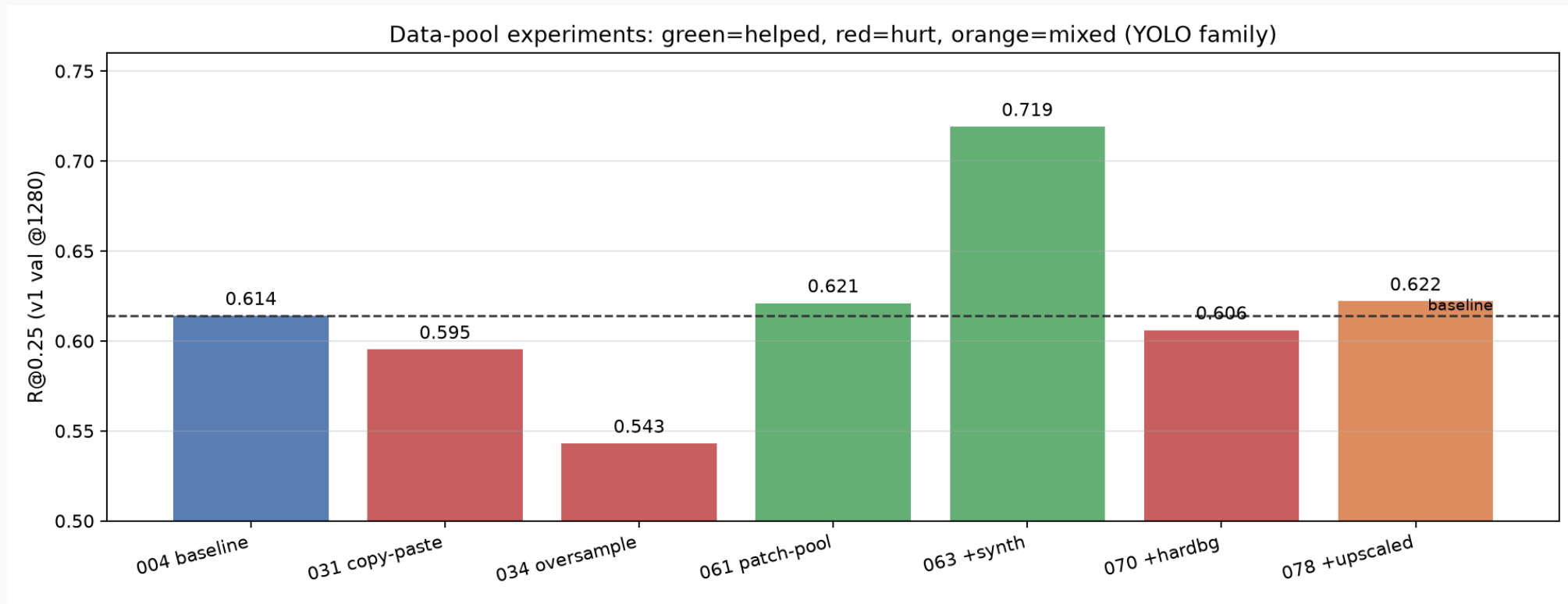
数据集重划分 · 划分如何影响结论可信度

划分	val 特点	暴露的问题
v1（最初）	181 图 / 377 GT，前缀序切分	bd/hei 每类 GT 少，类分布偏
v2	350 图 / 678 GT，类分层	bmss 127 实例独占 val ；与 v1 train 重叠 80.9%
v3（现行）	180 图，8:1:1 迭代分层	每类 val ≥ 8 实例，重合 7%—— 干净

- 划分带来的度量陷阱（实测）：
- 同一 yolo11s 在 v1 val 得 0.614、v2 val 得 0.532——**差 8.2pp 全来自划分，不是模型**
 - 不做泄漏审计的跨集对比全部作废

实践规范：每次重划分都跑泄漏检查脚本；每个 run 的 NOTES.md 存数据清单快照；v3 之后所有数字自治

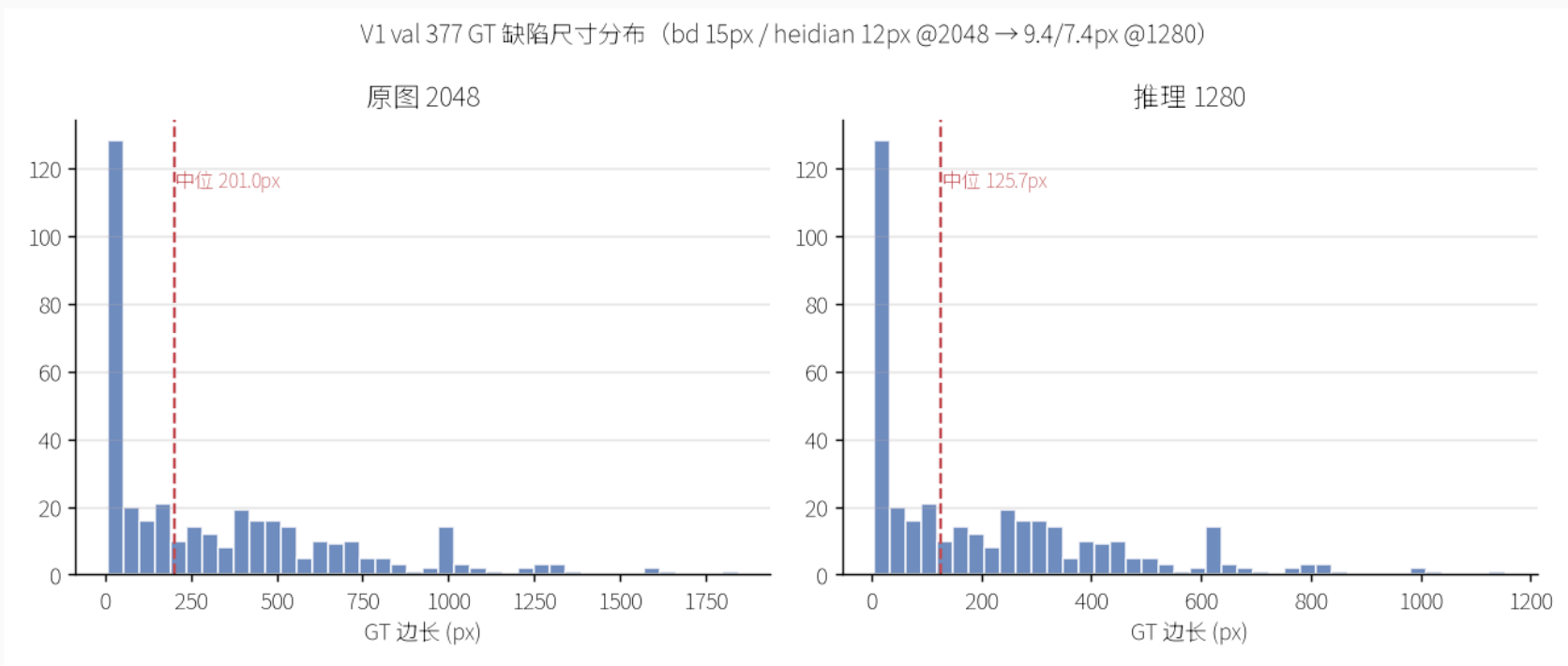
数据增强 · 池实验全景 (+10.5pp 与四次证伪)



正收益: 061 实例中心 patch (+0.7pp) → 063 +合成池 = **0.719** (+10.5pp, bd 0.477/hei 0.333) **证伪:** 070 难例背景 (0.606↓) / 078 放大池 (0.622, **11× 训练-推理尺度失配**) / 031 copy-paste / 034 过采样

规律: 增强图必须与推理尺度一致 (原位变体>放大重采样); FP 高不等于背景多 (val 标注稀疏藏真缺陷)

数据增强 · 尺度对齐是第一性原理



数据说话：v1 train bd 中位宽 0.0195 ($\approx 20\text{px}$)，val 仅 0.0063 ($\approx 7\text{px}$) —— **训练看到的 bd 比 val 大 3 倍**。这就是 bd/heidian 召回低的根因，任何增强池都必须回答“我的尺度分布是什么”。

由三个产物验证：

- V3-B 小尺度原位池（保尺度不放大）→ 079 验证
- DEIMv2 @1280 原生分辨率路线（103/104）→ bd 提到 **0.614**
- 078 放大池失败 = 反向验证尺度失配假设

工程维度：云端训练实践（本地 6GB → 云端）

Kaggle 双 T4 (095-078 时代):

- MCP 创建 (避免 P100 sm_60 不兼容) · device 0,1 · batch 32
- 132ep 实测 3.5h; QUICK_SAVE 防止推送自动起会话

4090 24G 单卡 (102-105 时代):

- DEIMv2 引擎改造: **梯度累积引擎补丁** (loss/accum, 按累积步进 optimizer+lr)
- batch4×累积 8=32 有效 batch, AMP + expandable_segments
- 断点续跑协议: checkpoint_freq + last.pth 语义验证

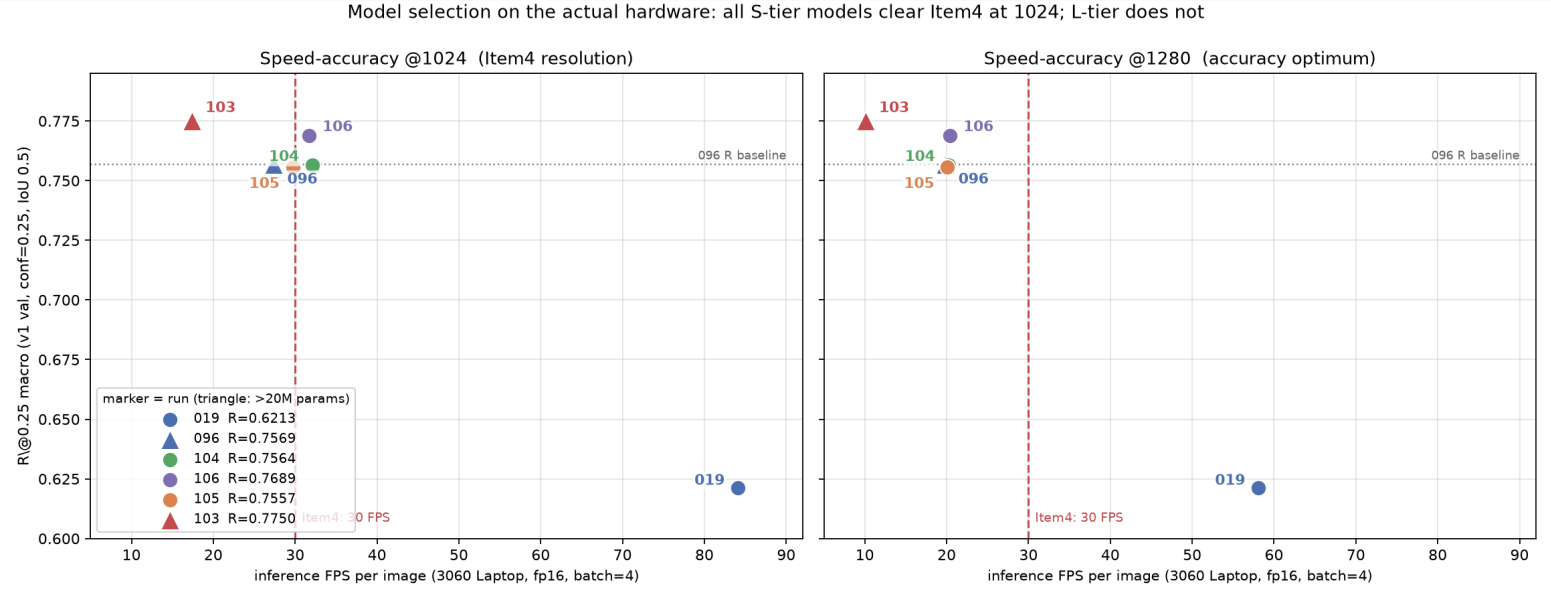
工程沉淀:

- 可复现: 每 run 存 config 快照 + 数据清单快照
- 可续跑: engine 支持任意批次断点恢复
- 可审计: NOTES.md 记录每次修复 (vitt 前缀 / num_classes / resume 语义)

终局：各维度对最终指标的贡献

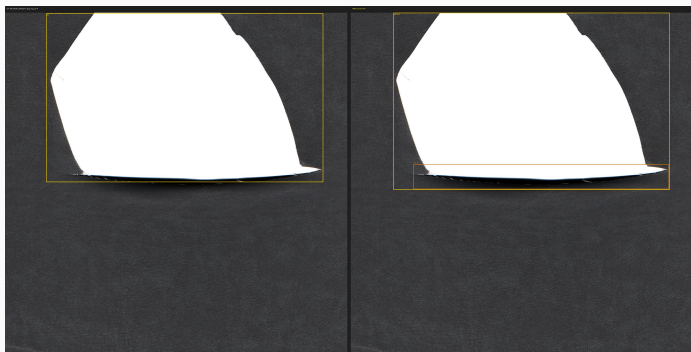
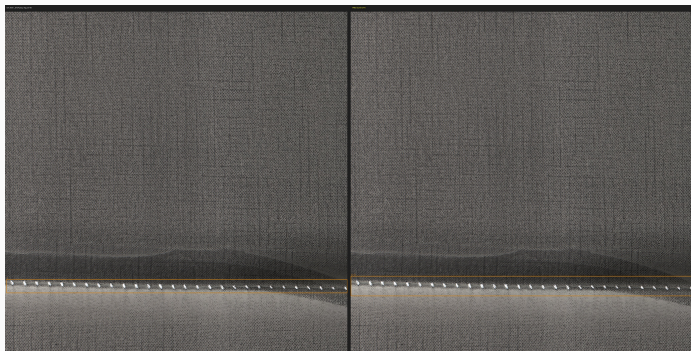
终局：各维度对最终指标的贡献

维度	贡献	证据
基线选型	底座 0.614	7 模型同配方对照
超参调整	+0.7pp	cls 加权 (019); 其余超参全负但排除成本最低
架构调整	+13.5pp	RT-DETR 范式 (096); 结构微创新全负
数据重划分	可信度保障	v2 泄漏作废 vs v3 干净 (079 起全部数字可复现)
数据增强	+10.5pp (063)	patch+合成池 @1280; 四种负形态排除
预训练表征	+5.6pp	063 0.719 → 103 0.775 (DEIMv2 DINOv3 微调)



终态模型检测效果 (左 GT / 右 预测, conf=0.47)

终态模型检测效果 (左 **GT** / 右 预测, **conf=0.47**)



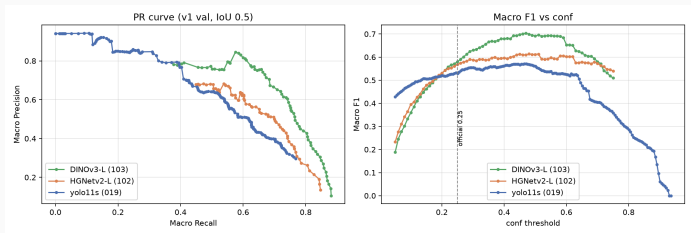
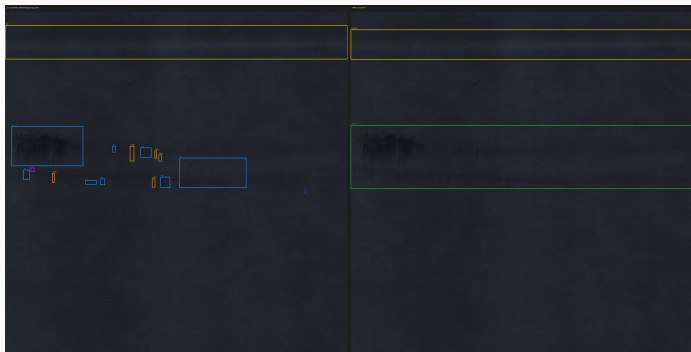
终态模型检测效果（左 GT / 右 预测，conf=0.47）



jt 接头完美检出 / pd 纸边阴影误检 / wy 小点类别混淆



终态模型检测效果 (左 GT / 右 预测, conf=0.47)



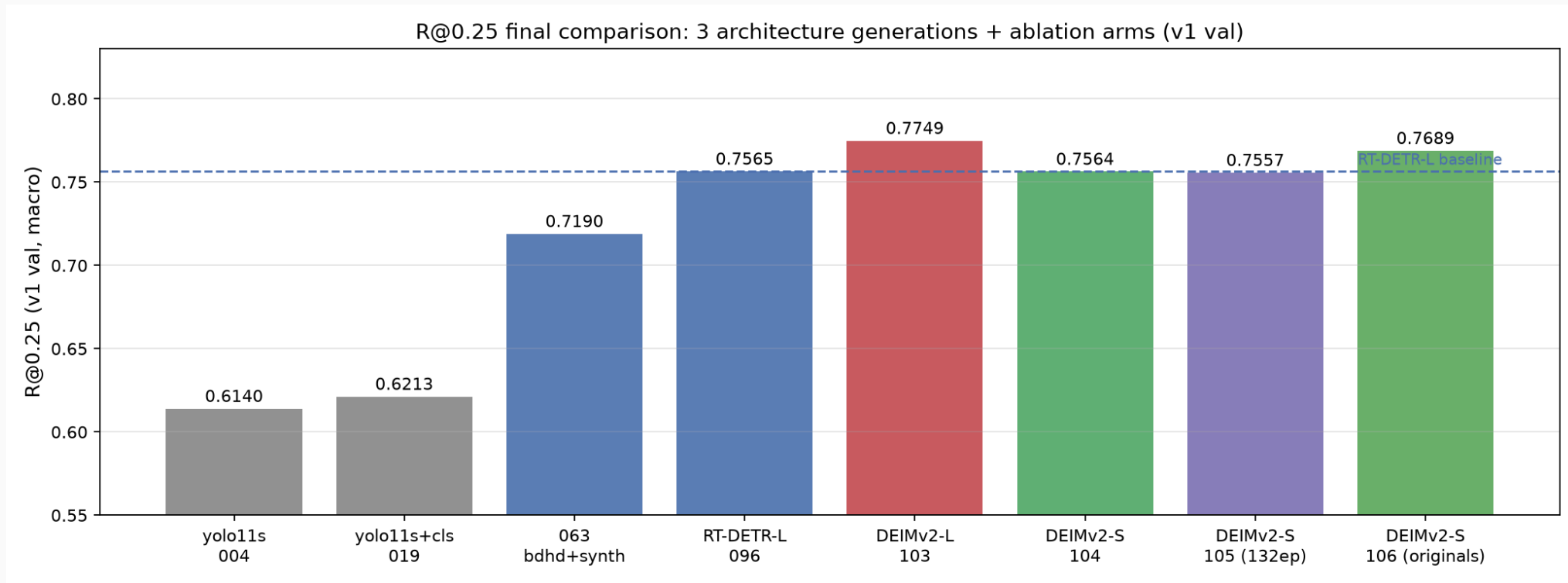
heidian+HD 横档对齐 / zmtly 碎片合并现象 / P-R 全曲线 (103 vs 102)

DEIMv2 系列：四轮完整实验

run	backbone	ep	R@0.25	P@0.25	FPS@1024 (3060 bs4)
102	HGNetv2-L（错误配方）	68	0.706	0.468	—
103	DINOv3-L（官方配方）	68	0.775	0.460	17.4 ✕
104	DINOv3-S（官方配方）	68	0.756	0.473	32.1 ✓
105	DINOv3-S（官方 132ep）	132	0.756	0.498	29.7
106	DINOv3-S（原图-only）	68	0.769	0.464	31.7 ✓

Item4 达标（本机 RTX 3060 Laptop 原生实测，fp16 batch=4，纯前向）：104/106（S 档 9.7M）@1024 = 32.1/31.7 FPS > 30，且 R 均超 RT-DETR 基线 0.757；103（L 档）0.775 为精度上限，17.4 FPS 需工程加速

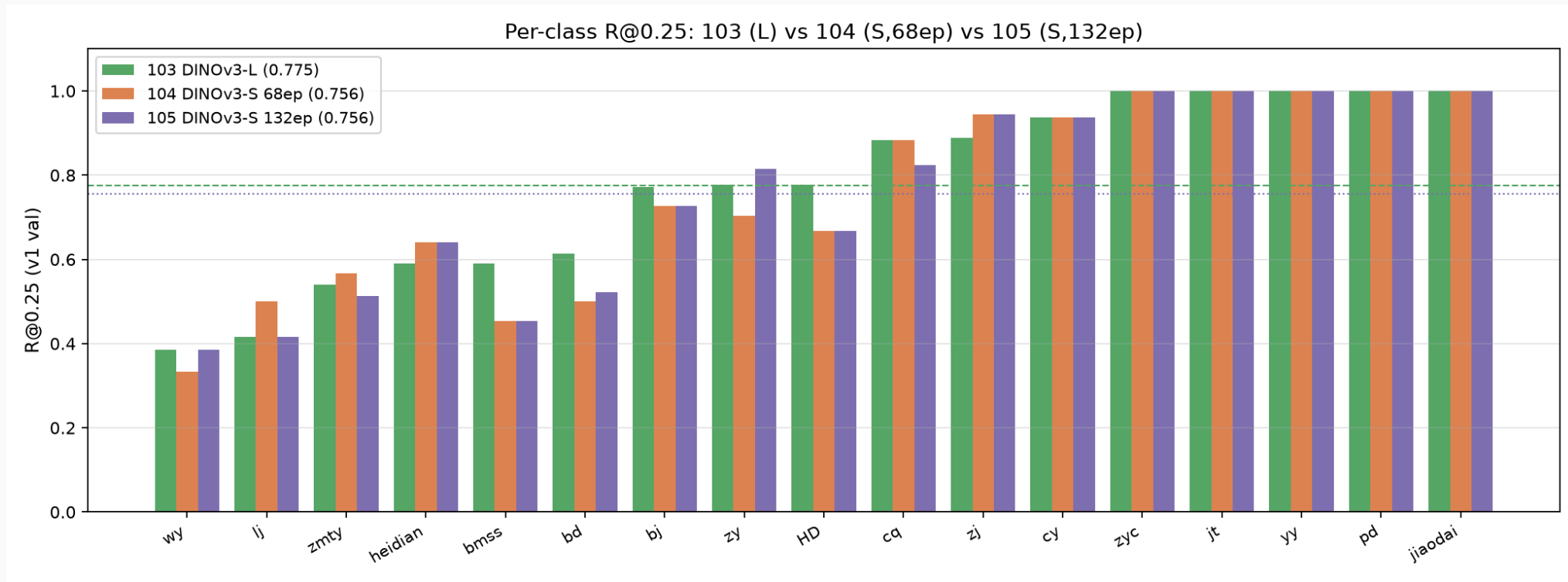
R@0.25 收官对比（含消融臂）



同口径（v1 val, conf=0.25, IoU 0.5）的全系列对比：从 yolo11s 基线 0.614 到 DEIMv2-L 0.775。

三个关键读数：①103（L）= 精度上限；②106（S，原图-only）= 消融最优，反超 104（S，混合池）；③104/105/106（S）= 速度达标交付版（1024 下 32.1/29.7/31.7 FPS），精度仍超 RT-DETR 基线。

103 / 104 / 105 逐类对照

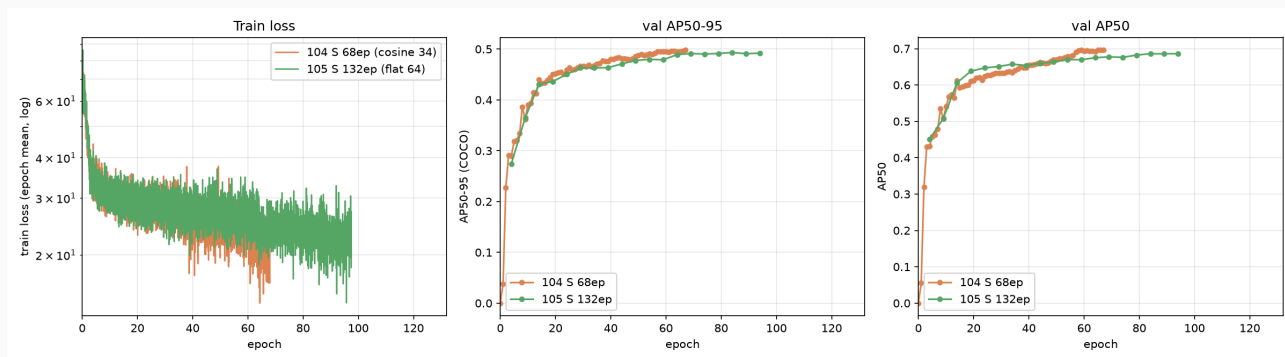


L vs S 的分工: L 在 bd (0.614 vs 0.500) / bmss (0.591 vs 0.455) 上明显更强——容量换小目标召回；S 在 heidian (0.641 vs 0.590) / zmt (0.568 vs 0.540) 上反超——蒸馏 backbone 的弱纹理判别不失真。

105 ≈ 104: 132ep 官方配方没有带来召回增益 (0.7557 vs 0.7564)，只把 P 从 0.473 提到 0.498。

消融一：训练时长不是杠杆（105 vs 104）

消融一：训练时长不是杠杆（105 vs 104）



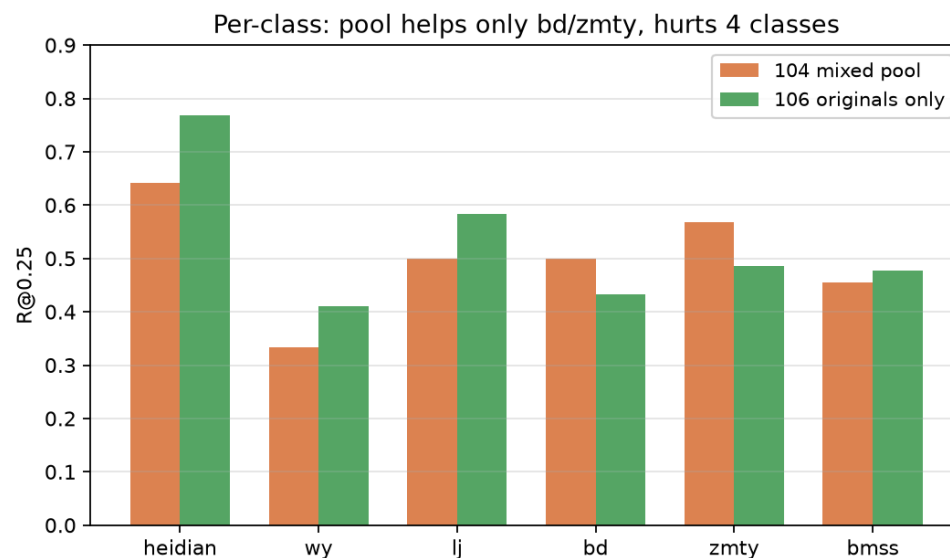
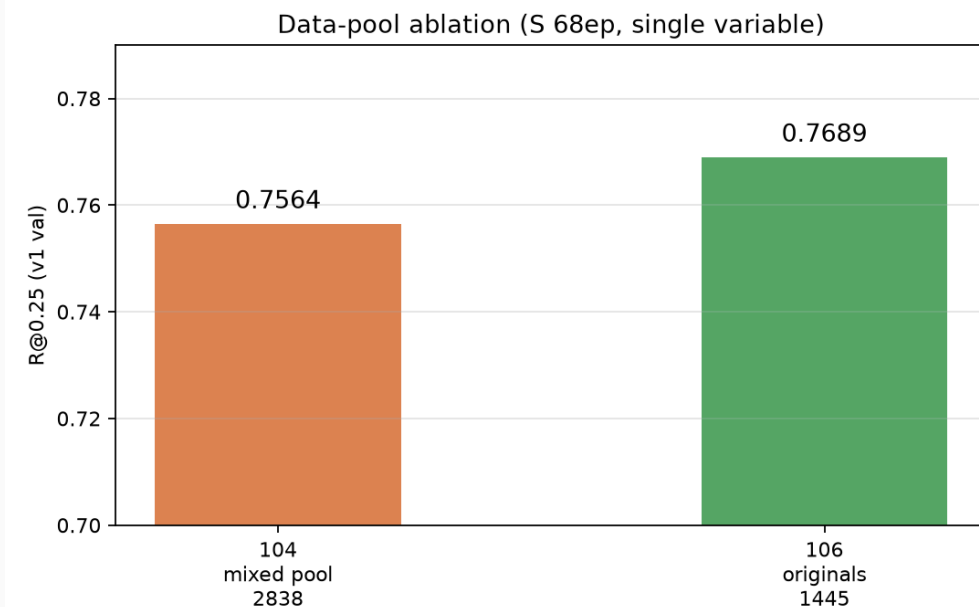
单变量：68ep cosine vs 132ep 官方 flat64，同数据同 backbone。

结果：

- 共同区间（ep0-58）两条 val 曲线几乎重合
- 104 的 cosine 尾段 +0.6pp 收敛到 0.498
- 105 的额外 64ep flat 段零增益，尾段 0.489-0.493

结论：68ep 已达 S 档配方上限，“欠训练”假设证伪。

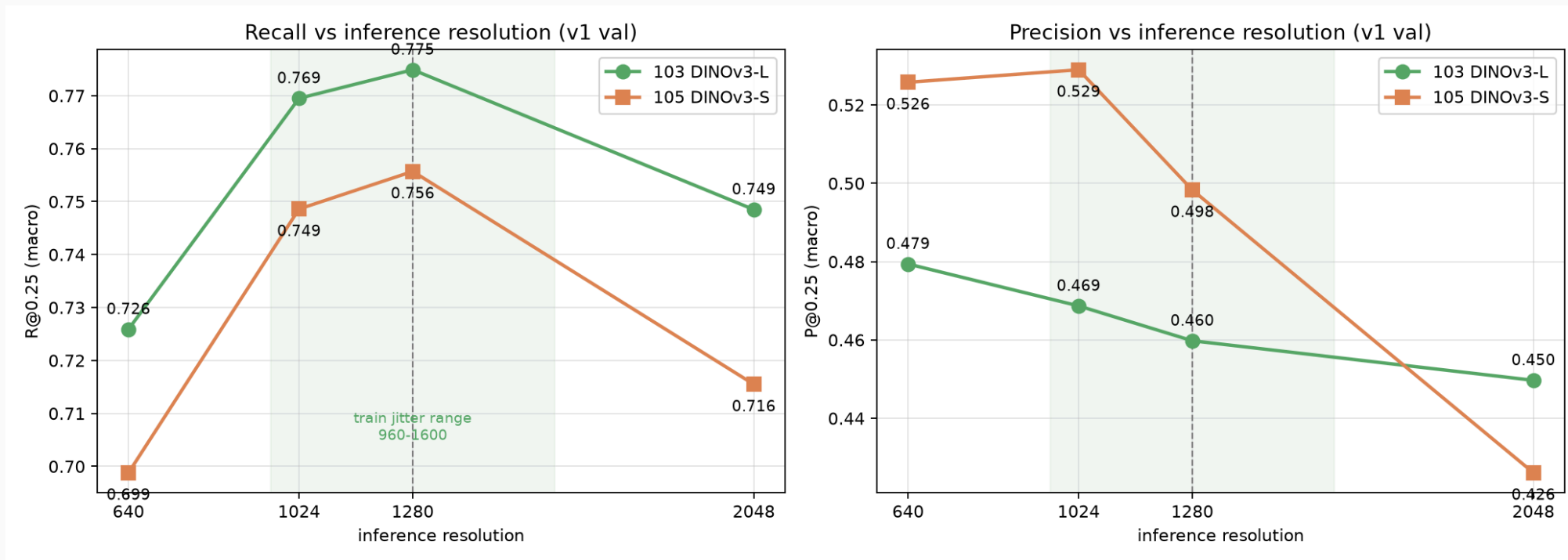
消融二：数据池（blanket 增强无效）



单变量：104 = 混合池 2838（原图 + 对比度池 + patch 池）vs 106 = 原图 1445，其余配方完全一致。

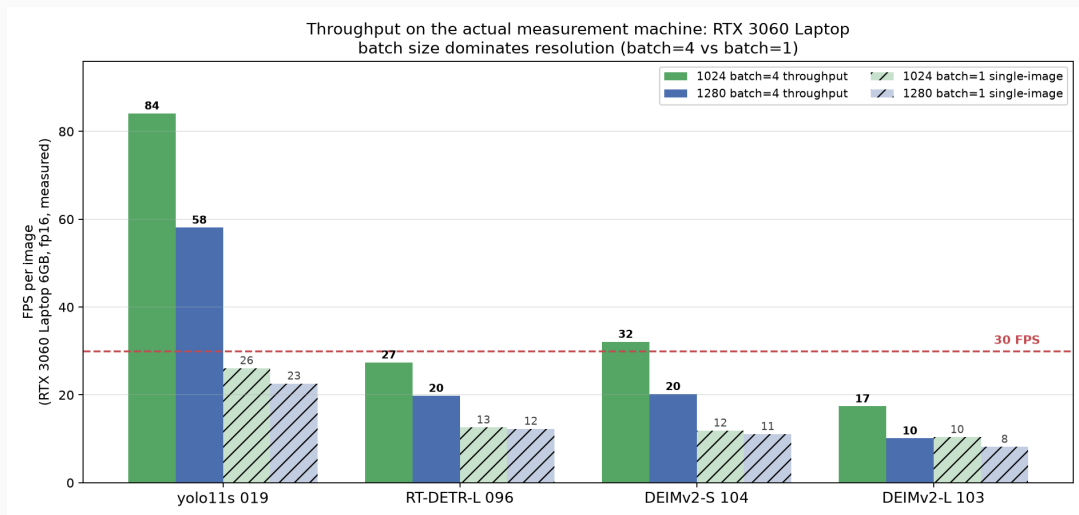
结果：去掉 1393 张池图后 R 反而 **+1.3pp** (0.7564→0.7689) ——池只对 bd/zmty 有正贡献，却让 heidian (-12.8pp) /wy/lj/zy 失血。**一刀切增强 = 增益与伤害互相抵消。**

消融三：推理分辨率（训练-推理尺度绑定）



在 val 原图 2048 尺度上重生成 anchors 后扫描四个推理分辨率：均为倒 U 型、顶点锁定 1280（训练尺度）。绿带 = 训练时的多尺度抖动范围 960-1600：内插（1024）仅掉 0.5-0.7pp，外插（640/2048）掉 2.6-5.7pp。结论：1280 不是妥协而是最优工作点；高/低分辨率推理均被数据否定。

速度评测方法学：口径决定结论



一次差点写反的结论。初测用 batch=1 单图计时，得 104 = 11.1 FPS → “Item4 不达标，需上 TensorRT”。

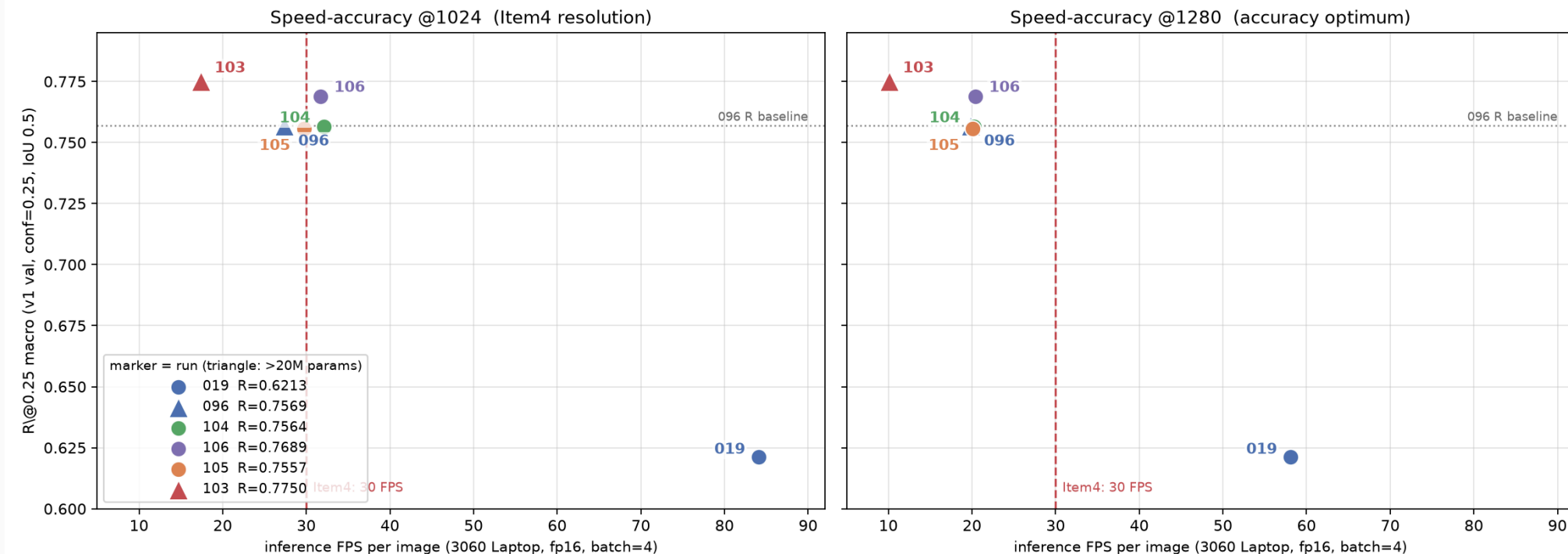
复核发现测量路径本身是变量：batch=1 每图重复付 Python/内核启动开销，GPU 空转；部署端是持续流。同权重、同 val 集、同机器改 batch=4 后：

- 104 @1280: 11.1 → **20.2** FPS (2.7×)
- 019 @1280: 22.6 → **58.1** FPS (3.2×)
- 096 @1280: 12.3 → **19.8** FPS (1.6×)

方法学结论：速度是与精度同级的口径敏感量（分辨率 / batch / 纯前向 vs 端到端 / 硬件），与评测纪律同源——**先固化口径，再比较模型。**

速度-精度权衡：硬件现实与交付点

Model selection on the actual hardware: all S-tier models clear Item4 at 1024; L-tier does not



左 (@1024, Item4 分辨率)：S 档三个模型 (104/106/105) 在 30 FPS 线上方，L 档 103 (17.4) 与 096 (27.4) 在其下；右 (@1280, 精度最优)：全部模型落到 30 FPS 线下一——分辨率与速度的取舍必须显式量化，不能靠折算。

过去报告的“31.9 FPS@3080 (×2.5 折算)”是 batch=1 口径 + 跨卡外推的双重近似；本页为 3060 本机原生 batch=4 实测，判定不再依赖外推。

交付点：104 (S, R 0.756) 为 Item1×Item4 帕累托点；106 (原图-only, R 0.769) 精度更高且同样过线，是消融给出的备选。

结论

1. **五维度系统探究**: 基线选型 → 超参 → 架构 → 划分 → 增强, 每条路径都有对照实验和量化结论
2. **最大的三个杠杆**: 范式转换 (RT-DETR +13.5pp)、DINOv3 预训练表征 (+5.6pp)、尺度对齐的数据池 (+10.5pp)
3. **最终交付**: 104 (DINOv3-S, R@0.25 0.756, 3060 本机 1024 实测 32.1 FPS) **精度与速度双达标**; 103 (L 档) 0.775 为精度上限 (17.4 FPS, 需工程加速)
4. **三个消融结论**: 训练时长无用 (132ep 零增益)· blanket 数据池负收益 (原图反超 +1.3pp)· 推理尺度必须绑定 1280
5. **口径即结论**: 速度的三个变量 (batch / 分辨率 / 前向-端到端) 可让同一模型差 3.2×; 旧 “31.9 FPS@3080 折算” 已由本机原生 batch=4 实测替代
6. **负结果同样有价值**: 注意力 ×9、DINOv3 融合 ×8、背景抑制 ×4、blanket 池 ×1——排除错误方向本身就是工作量
7. **工程纪律**: 无泄漏划分 / 单变量验证 / 配方快照 / 可续跑引擎——所有数字可复现